

Radiografías con contexto

Tres objetos **de tu vida**, modelados como clases de Java, con cada parte etiquetada y funcionando. No se trata de escribir mucho código: se trata de que puedas **justificar** cada atributo y cada método, y de que uses de verdad los tipos y operadores de la clase del 7 de septiembre.

En una línea

Tres clases, cada una con su comentario de modelado de 4 líneas, sus partes etiquetadas y su `main` que imprime la tarjeta del objeto. Entre las tres deben aparecer **4 tipos distintos** y **todos los operadores** que vimos. Se entrega el `.java` y el `.class`.

Parte 0 · Qué se entrega y cómo se califica

QUÉ	DETALLE
Cuándo	Lunes 14 de septiembre , antes de la clase.
Dónde	Su rama <code>entregas_apellido_nombre</code> · carpeta <code>entregas/apellido_nombre/tareas/t01/</code> . Solo push. No se abre pull request.
Qué archivos	Por cada objeto: el <code>.java</code> y su <code>.class</code> (compilado con JDK 21), más un <code>QUIZ.md</code> . Son 7 archivos , más nada.
Cómo se califica	Justificación del modelo 30 % · código completo y etiquetado 30 % · que corra 20 % · quiz de vocabulario 20 % .

Lo que hace que la tarea valga la mitad

Yo ejecuto tu `.class`, no lo recompilo. Si no corre en mi máquina, la tarea está a la mitad aunque el `.java` esté perfecto. Antes de subirlo: borra el `.class`, vuelve a compilar con `javac` y córrelo con `java` una última vez.

Parte 1 · Los tres objetos

Objetos **de tu vida**: cosas que usas, que tienes o que ves todos los días. La regla es que puedas contar en qué programa vivirían y para qué serviría ese programa.

Prohibidos

`Circulo`, `Coche`, `CuentaBancaria`, `Cancion`, `Alumno`, `Termometro`, `Videojuego` y cualquiera de las prácticas. Ya los modelamos en clase; la tarea es que modelés tú.

Menú de ideas, por si no se te ocurre nada

No es obligatorio usarlas: son para arrancar. La última columna dice qué operador practica cada una, para que armes tus tres cubriendo todo lo que se pide.

OBJETO	QUÉ SABE	QUÉ SABE HACER	PRACTICA
Mochila	<code>double pesoKg</code> · <code>int</code> <code>objetos</code> · <code>boolean</code> <code>cerrada</code>	<code>meter(double kg)</code> void · <code>estaPesada()</code> boolean	<code>+=</code> · <code>>=</code>
Celular	<code>double bateria</code> · <code>int</code> <code>apps</code> · <code>boolean</code> <code>silencio</code>	<code>usar(double min)</code> void · <code>necesitaCarga()</code> boolean	<code>-</code> · <code><</code> · <code> </code>
Cafetera	<code>int mlAgua</code> · <code>int</code> <code>tazasServidas</code>	<code>servir()</code> void · <code>alcanzaPara(int tazas)</code> boolean	<code>/</code> entera · <code>%</code>
Serie	<code>String titulo</code> · <code>int</code> <code>vistos</code> · <code>int total</code>	<code>ver(int caps)</code> void · <code>porcentajeVisto()</code> double	casting · división <code>double</code>

Bicicleta	<code>int cambio</code> · <code>double</code> <code>velocidad</code> · <code>boolean</code> <code>luz</code>	<code>subirCambio()</code> <code>void</code> · <code>vaSegura()</code> <code>boolean</code>	<code>++</code> · <code><=</code> · <code>&&</code>
Taquería	<code>int tacos</code> · <code>double</code> <code>precio</code> · <code>char salsa</code>	<code>pedir(int n)</code> <code>void</code> · <code>cambioDe(double paga)</code> <code>double</code>	<code>*</code> · <code>-</code> · <code>char</code>
Ruta de camión	<code>int pasajeros</code> · <code>double</code> <code>tarifa</code> · <code>boolean</code> <code>enParada</code>	<code>subir(int n)</code> <code>void</code> · <code>recaudado()</code> <code>double</code>	<code>*</code> · <code>+=</code> · <code>==</code>
Impresora	<code>int hojas</code> · <code>int</code> <code>copiasHechas</code> · <code>char</code> <code>tinta</code>	<code>imprimir(int n)</code> <code>void</code> · <code>tieneHojas()</code> <code>boolean</code>	<code>-</code> · <code>></code> · <code>%</code>

Parte 2 · Qué debe traer cada clase

1 El comentario de modelado, 4 líneas, arriba de la clase

CONTEXTO (¿en qué programa vive este objeto?) · **QUÉ SABE Y POR QUÉ** (cada atributo con su razón) · **QUÉ SABE HACER Y POR QUÉ** (por qué `void` o por qué retorna, y por qué esos parámetros) · **QUÉ IGNORE** (1 o 2 cosas del objeto real que no modelaste, y por qué).

2 La clase, con cada parte etiquetada

Mínimo **2 atributos** y **2 métodos**: uno `void` con parámetro y uno que retorne. Cada atributo con su comentario de tipo, nombre y valor inicial; cada método con su **firma** en comentario: `// firma: meter(double)`.

3 Un constructor, en una de las tres clases

Elige la clase donde tenga más sentido: aquella que **no debería poder nacer vacía**. Mismo nombre que la clase, **sin** tipo de retorno, y adentro `this.atributo =`

`parametro;` . En las otras dos no hace falta. Si te animas a poner un segundo constructor vacío en la misma clase, eso es **sobrecarga** y cuenta como punto extra.

```
// firma: Mochila(double) · sin tipo de retorno
Mochila(double pesoKg) {
    this.pesoKg = pesoKg; // this.pesoKg = ATRIBUTO · pesoKg = PAR
}
```

Ojo: en cuanto escribes ese constructor, `new Mochila()` deja de existir. En tu `main` tienes que crear el objeto con sus datos: `new Mochila(3.5)`.

4 El `main` completado: la tarjeta del objeto

Crear el objeto, imprimir el **estado inicial**, invocar los métodos diciendo su firma, y volver a imprimir el **estado final**. Eso demuestra que el objeto cambia como dijiste que iba a cambiar.

Y entre los tres objetos, esta lista de tipos y operadores

No tienen que estar todos en la misma clase: repártelos entre los tres. Marca cada uno con un comentario corto donde lo uses, así: `// operador %`.

DEBE APARECER	CUÁNTAS VECES	EJEMPLO
Tipos distintos	al menos 4 de: <code>int</code> , <code>double</code> , <code>boolean</code> , <code>char</code> , <code>String</code>	<code>double bateria;</code> · <code>char salsa;</code>
Aritméticos	los cinco: <code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	<code>total = tacos * precio;</code>
Abreviado o contador	al menos uno: <code>++</code> , <code>--</code> o <code>+=</code>	<code>copiasHechas++;</code>
Relacionales	al menos dos distintos	<code>return hojas > 0;</code>
Lógico	al menos uno: <code>&&</code> , <code> </code> o <code>!</code>	<code>return vidas > 0 && !pausado;</code>

Casting	uno explícito	<code>int completas = (int) horas;</code>
Constructor	uno, con parámetros, en una de las tres clases	<code>Mochila(double pesoKg) { this.pesoKg = pesoKg; }</code>
Punto extra	una constante <code>static final</code> , un contador <code>static</code> o un segundo constructor (sobrecarga)	<code>static final int PUNTOS_POR_NIVEL = 1000;</code>

La trampa de la división

Si divides `int` entre `int`, el resultado es `int` y los decimales se van sin avisar:

`90 / 60` da **1**, no 1.5. Si quieres decimales, uno de los dos tiene que ser `double`:

`90 / 60.0` da **1.5**. Es el error que más va a salir en esta tarea.

Parte 3 · Un ejemplo resuelto, de principio a fin

Este es `Videojuego`, y por eso mismo **ya no lo puedes entregar**. Léelo como muestra de qué tan lejos hay que llegar, no como plantilla para rellenar.

```
// CONTEXT0: app que lleva el avance de una partida en mi consola.
// QUE SABE Y POR QUE: titulo (para mostrarlo), puntos y vidas (definen s
//             horasJugadas (para saber si rindo), pausado (no cu
//             dificultad (una sola letra: F, N o D).
// QUE SABE HACER Y POR QUE: anotar(int) es void porque solo cambia el es
//             la pantalla lo necesita; puedeSeguir() retorna boo
// QUE IGNORE: los graficos y el nombre del jugador: no cambian nada de T
// CONSTRUCTOR: titulo y dificultad se piden al crear la partida, porque
//             existe; los demas atributos si tienen valor por defecto n
public class Videojuego {

    // ===== CONSTANTE (opcional, punto extra) =====
    static final int PUNTOS_POR_NIVEL = 1000;    // tipo: int · nombre en

    // ===== ATRIBUTOS =====
```

```

String titulo = "Sin nombre";    // tipo: String    · objeto, no primitivo
int puntos = 0;                  // tipo: int        · valor inicial: 0
int vidas = 3;                   // tipo: int        · valor inicial: 3
double horasJugadas = 0.0;       // tipo: double    · valor inicial: 0.0
boolean pausado = false;         // tipo: boolean  · valor inicial: false
char dificultad = 'N';           // tipo: char      · N de normal

// ===== CONSTRUCTOR =====
// firma: Videojuego(String, char) · mismo nombre de la clase · SIN tilde
Videojuego(String titulo, char dificultad) {
    this.titulo = titulo;          // this.titulo es el ATRIBUTO, tilde
    this.dificultad = dificultad;
}

// ===== METODOS =====
// firma: anotar(int) · retorno: void · parametro: int gana
void anotar(int gana) {
    puntos += gana;                // operador +=
}

// firma: perderVida() · retorno: void · sin parametros
void perderVida() {
    vidas--;                       // operador --
}

// firma: jugar(double) · retorno: void · parametro: double horas
void jugar(double horas) {
    horasJugadas = horasJugadas + horas; // operador +
}

// firma: nivel() · retorno: int · sin parametros
int nivel() {
    return puntos / PUNTOS_POR_NIVEL;    // division ENTERA: 2500/100 = 25
}

// firma: esNivelPar() · retorno: boolean · sin parametros
boolean esNivelPar() {

```

```

        return nivel() % 2 == 0;                // operadores % y ==
    }

    // firma: puedeSeguir() · retorno: boolean · sin parametros
    boolean puedeSeguir() {
        return vidas > 0 && !pausado;          // operadores >, && y !
    }

    // firma: puntosPorHora() · retorno: double · sin parametros
    double puntosPorHora() {
        return puntos / horasJugadas;          // int entre double: da do
    }

    public static void main(String[] args) {
        System.out.println("RADIOGRAFIA: Videojuego");

        Videojuego v = new Videojuego("Metroid", 'D');    // el constructo

        System.out.println("[estado inicial]");
        System.out.println("titulo      = " + v.titulo);
        System.out.println("puntos      = " + v.puntos);
        System.out.println("vidas       = " + v.vidas);
        System.out.println("horasJugadas = " + v.horasJugadas);
        System.out.println("pausado     = " + v.pausado);
        System.out.println("dificultad  = " + v.dificultad);

        System.out.println("[invocando metodos]");
        v.anotar(2500);
        System.out.println("anotar(int) -> void; puntos = " + v.puntos);
        v.jugar(3.5);
        System.out.println("jugar(double) -> void; horasJugadas = " + v.horasJugadas);
        v.perderVida();
        System.out.println("perderVida() -> void; vidas = " + v.vidas);
        System.out.println("nivel() -> int; " + v.nivel());
        System.out.println("esNivelPar() -> boolean; " + v.esNivelPar());
        System.out.println("puedeSeguir() -> boolean; " + v.puedeSeguir());
        System.out.println("puntosPorHora() -> double; " + v.puntosPorHora());
    }
}

```

```

        int horasCompletas = (int) v.horasJugadas;    // casting: 3.5 -> 3
        System.out.println("casting (int) 3.5 = " + horasCompletas);

        System.out.println("[estado final]");
        System.out.println("puntos = " + v.puntos + " | vidas = " + v.vidas
                           + " | horasJugadas = " + v.horasJugadas);
    }
}

```

Se compila y se corre así, y esto es lo que imprime de verdad:

```

javac Videojuego.java    # nace Videojuego.class
java Videojuego          # SIN el .class al final

```

SALIDA REAL

```

RADIOGRAFIA: Videojuego
[estado inicial]
titulo      = Metroid
puntos      = 0
vidas       = 3
horasJugadas = 0.0
pausado     = false
dificultad  = D
[invocando metodos]
anotar(int) -> void; puntos = 2500
jugar(double) -> void; horasJugadas = 3.5
perderVida() -> void; vidas = 2
nivel() -> int; 2
esNivelPar() -> boolean; true
puedeSeguir() -> boolean; true
puntosPorHora() -> double; 714.2857142857143
casting (int) 3.5 = 3
[estado final]
puntos = 2500 | vidas = 2 | horasJugadas = 3.5

```

Dónde quedó cada cosa de la lista, en este ejemplo

Tipos: `String`, `int`, `double`, `boolean` y `char`, cinco de cinco. **Aritméticos:** `+` en `jugar`, `-` escondido en `--`, `/` en `nivel` y en `puntosPorHora`, `%` en `esNivelPar`; el `*` es el único que este ejemplo no trae, y por eso tú sí lo necesitas en alguno de tus tres. **Abreviados:** `+=` y `--`. **Relacionales:** `==` y `>`. **Lógicos:** `&&` y `!`. **Casting:** `(int) v.horasJugadas`. **Constructor:** `Videojuego(String, char)`, que es justamente el que se pide en una de tus tres. **Punto extra:** la constante `PUNTOS_POR_NIVEL`.

Dos detalles del ejemplo que valen tarea

`nivel()` usa división **entera** a propósito: con 2500 puntos y 1000 por nivel, el nivel es 2, no 2.5. En cambio `puntosPorHora()` divide un `int` entre un `double` y por eso sí sale con decimales. Los dos son correctos porque cada uno responde una pregunta distinta; lo que se califica es que sepas cuál usaste y por qué.

Parte 4 • El esqueleto del que partes

Está en el repo, en `tareas/t01/plantillas/MiObjeto.java`. Cópialo tres veces, uno por objeto, y renombra la clase **y el archivo** cada vez. Al lado está `tareas/t01/plantillas/QUIZ.md`, que es el de la Parte 5.

```
// CONTEXT0: (¿en que programa vive este objeto? una linea)
// QUE SABE Y POR QUE: (cada atributo, con su razon)
// QUE SABE HACER Y POR QUE: (cada metodo: por que void o por que retorna
//                                y por que esos parametros)
// QUE IGNORE: (1 o 2 cosas del objeto real que no modelaste, y por que)
public class MiObjeto {    // TODO: renombra la clase Y el archivo (Pascal

    // ===== ATRIBUTOS (etiqueta cada parte) =====
    // ej:  double saldo = 0;    // tipo: double · nombre: saldo · valor i

    // ===== CONSTRUCTOR (obligatorio en UNA de tus tres clases) =====
    // Mismo nombre que la clase, SIN tipo de retorno (ni void).
    // Pon aqui los datos sin los que el objeto no tiene sentido; el rest
```

```

// ej: // firma: MiObjeto(double) · sin tipo de retorno
//      MiObjeto(double saldo) {
//          this.saldo = saldo; // this.saldo = ATRIBUTO · saldo a
//      }
// OJO: si escribes este, new MiObjeto() deja de existir. O lo llamas
//      o escribes tambien el constructor vacio: MiObjeto() { }

// ===== METODOS (etiqueta cada parte y su firma) =====
// ej: // firma: retirar(double) · retorno: void · parametro: double

public static void main(String[] args) {
    System.out.println("RADIOGRAFIA: MiObjeto");
    // 1: crea tu objeto (con new MiObjeto(...)) si le pusiste constru
    System.out.println("[estado inicial]"); // 2: imprime cada atr
    System.out.println("[invocando metodos]"); // 3: firma + invocaci
    System.out.println("[estado final]"); // 4: imprime atributo
}
}

```

Parte 5 · El quiz de vocabulario

Un archivo `QUIZ.md`, en la misma carpeta que tus tres clases. Diez preguntas cortas sobre lo que vimos el lunes. Vale **20 % de la tarea**, y es la parte que más rápido se hace si de verdad entendiste, y la que más se nota si no.

Las cuatro reglas

- 1. Con tus palabras.** Si la respuesta es la definición de la slide copiada o una de internet pegada, esa pregunta vale cero. No busco la definición correcta, busco la tuya.
- 2. Máximo 3 renglones por respuesta.** Se califica que se entienda, no que sea larga.
- 3. Donde diga "de tu tarea" pega la línea real de tu código** no un ejemplo inventado.
- 4. Una respuesta razonada pero equivocada da más puntos que una copiada perfecta**, porque la primera se corrige en clase y la segunda no me dice nada.

Las diez preguntas

PREGUNTA

- 1 ¿Qué es un **atributo**? ¿En qué se diferencia de una variable cualquiera? Pon un ejemplo de cada uno **sacado de tu tarea**.
- 2 ¿Qué es un **constructor**, en qué momento exacto se ejecuta y cuántas veces? Pega el tuyo y di qué habría pasado con ese objeto si no lo hubieras escrito.
- 3 ¿Para qué sirve `this` adentro de un constructor? ¿Qué pasa si se te olvida: truena, o pasa algo peor?
- 4 ¿Qué es la **firma** de un método? Escribe la firma de dos métodos tuyos, tal como se escriben.
- 5 **Parámetro y argumento**: ¿cuál es cuál? Pon una línea de tu código donde se vea el parámetro y otra donde se vea el argumento.
- 6 ¿Qué significa que un método sea `void`? Viendo tu propio código: ¿cómo decidiste cuál era `void` y cuál tenía que retornar?
- 7 ¿Qué es un tipo **primitivo** y en qué se diferencia de una **referencia**? ¿Por qué un `int` nunca puede valer `null` y un `String` sí?
- 8 ¿Por qué en Java `7 / 2` da `3` y no `3.5`? ¿Qué es lo mínimo que cambiarías para que diera `3.5`?
- 9 ¿Qué hace un **casting** como `(int)` y qué se pierde al hacerlo? Pega la línea de tu tarea donde lo usaste y di qué se perdió ahí.
- 10 ¿Qué hace `new`? ¿Cuál es la diferencia entre la **clase** y una **instancia**, dicha con uno de tus tres objetos?

El formato del archivo

Cópialo tal cual y contesta debajo de cada pregunta. Se llama `QUIZ.md`, con esas mayúsculas.

```
# QUIZ T01 · Nombre Apellido
```

```
## 1. Atributo contra variable
```

(tu respuesta, maximo 3 renglones)

2. Constructor: que es, cuando corre, cuantas veces

(tu respuesta)

```
```java
```

```
// pega aqui tu constructor
```

```
```
```

3. Para que sirve this

(tu respuesta)

4. La firma de un metodo

(tu respuesta, con tus dos firmas)

5. Parametro y argumento

(tu respuesta, con una linea de cada uno)

6. Que significa void

(tu respuesta)

7. Primitivo contra referencia, y el null

(tu respuesta)

8. Por que $7 / 2$ da 3

(tu respuesta)

9. El casting y lo que se pierde

(tu respuesta, con tu linea)

10. new, clase e instancia

(tu respuesta)

Cómo se ve una respuesta que sí cuenta

Pregunta 1, mal: "Un atributo es una variable declarada dentro de una clase que representa el estado del objeto." Eso está bien escrito y no me dice si entendiste.

Pregunta 1, bien: "El atributo es lo que el objeto se acuerda aunque nadie lo esté usando. En mi `Mochila`, `pesoKg` es atributo porque la mochila sigue pesando lo mismo cuando termina el método. En cambio el `kg` de `meter(double kg)` es una variable que existe nada más mientras corre ese método y luego desaparece."

Parte 6 • Cómo se entrega

Cambió respecto a la P0: ya no se abren pull requests

Cada quien tiene **una sola rama para todo el semestre**:

`entregas_apellido_nombre`. Todas las tareas del semestre viven ahí, cada una en su carpeta. **No se abre pull request**: con que tu `git push` llegue, está entregado. Yo leo tu rama directo.

```
entregas/  
└─ apellido_nombre/  
    └─ tareas/  
        └─ t01/  
            ├── Mochila.java      # tus nombres, no estos  
            ├── Mochila.class  
            ├── Celular.java  
            ├── Celular.class  
            ├── Cafetera.java  
            ├── Cafetera.class  
            └─ QUIZ.md
```

```
# 1. parate en TU rama (la misma de todo el semestre)  
git checkout entregas_apellido_nombre  
git pull  
  
# si es la primera vez y todavia no existe:  
# git checkout -b entregas_apellido_nombre
```

```
# 2. la carpeta de esta tarea
mkdir -p entregas/apellido_nombre/tareas/t01
# ahí van tus 3 .java, tus 3 .class y tu QUIZ.md

# 3. guarda y sube
git add entregas/apellido_nombre/tareas/t01
git commit -m "T01: tres radiografías con contexto"
git push -u origin entregas_apellido_nombre
```

Los tres errores de entrega que sí cuestan puntos

1. Subir a `main` en vez de a tu rama. Antes de hacer `git add`, corre `git branch` y confirma que el asterisco esté en `entregas_apellido_nombre`. 2. Poner la carpeta en otro lado. Es `entregas/apellido_nombre/tareas/t01/`, exactamente así, en minúsculas. 3. Hacer `commit` y no hacer `push`: si no está en GitHub, no existe. Ábrelo en el navegador y compruébalo tú.

✓ Checklist antes del último push

☐ Tres `.java`, tres `.class` y el `QUIZ.md`: siete archivos, nada más ☐ Todo dentro de `entregas/apellido_nombre/tareas/t01/` ☐ `git branch` muestra el asterisco en mi rama, no en `main` ☐ Cada clase con su comentario de modelado de 4 líneas ☐ Cada atributo y cada método etiquetados, con la firma en comentario ☐ Los tres `main` imprimen estado inicial, invocaciones y estado final ☐ Una de las tres clases tiene constructor con parámetros, y su `main` la crea con sus datos ☐ Entre los tres: 4 tipos, los cinco aritméticos, un abreviado, dos relacionales, un lógico y un casting ☐ Borré los `.class`, recompilé y corrí los tres una última vez ☐ Ningún objeto de la lista de prohibidos ☐ Hice `push` y lo verifiqué abriendo mi rama en GitHub

Si algo falla

SÍNTOMA

QUÉ PASÓ

| | |
|---|---|
| <code>class X is public, should be declared in a file named X.java</code> | Renombraste la clase pero no el archivo, o al revés. Los dos se llaman igual, con la misma mayúscula. |
| <code>non-static variable x cannot be referenced from a static context</code> | Usaste un atributo directo dentro de <code>main</code> . Primero crea el objeto: <code>Mochila m = new Mochila();</code> y luego <code>m.pesoKg</code> . |
| <code>missing return statement</code> | El método promete un tipo y no devuelve nada. Calcularlo no basta: falta el <code>return</code> . |
| <code>incompatible types: possible lossy conversion from double to int</code> | Le estás dando un decimal a un <code>int</code> . O cambias el tipo, o pones el casting: <code>(int)</code> . |
| La división te da <code>1</code> y esperabas <code>1.5</code> | <code>int</code> entre <code>int</code> da <code>int</code> . Haz que uno sea <code>double</code> : <code>90 / 60.0</code> . |
| <code>constructor Mochila in class Mochila cannot be applied to given types;</code> | Escribiste un constructor con parámetros y luego llamaste <code>new Mochila()</code> . El vacío ya no existe: llámalo con sus datos, o escribe también <code>Mochila() { }</code> . |
| El constructor corre pero el atributo se queda en <code>0.0</code> | Te faltó el <code>this</code> : escribiste <code>pesoKg = pesoKg;</code> y se asignó a sí mismo. Compila sin quejarse. Va <code>this.pesoKg = pesoKg;</code> . |
| Tu constructor nunca corre | Le pusiste tipo de retorno: <code>void Mochila(double kg)</code> . Eso es un método, no un constructor. Quítale el <code>void</code> . |
| <code>Error: Could not find or load main class X.class</code> | Corriste <code>java X.class</code> . Es <code>java X</code> , sin el <code>.class</code> . |

MPOO 2027-1 · Grupo 6 · Prof. Oscar Ruiz. Tarea 1, entrega el lunes 14 de septiembre. Lo que se califica es que puedas justificar el modelo, que el código esté completo y etiquetado, y que corra. Si

te atorras, escribe en la Discussion del repo: es mejor preguntar el miércoles que entregar a medias el lunes.